SafeBite — Food Safety Transparency Platform

Comprehensive Project Implementation, Verification & Evidence Dossier

Status: Production Ready (Stabilized)	**Target Environment:** Local / Docker Containerized
Backend API: FastAPI / SQLAlchemy 2.0 (Port 8000)	**Frontend:** React 18 / Vite SPA (Port 5173 / 3000)
Test Coverage: 23/23 Pytest Green (100%)	**Mutation Score:** 100% (7/7 Mutants Killed)
Data Provenance: Official FSSAI/NYC + Synthetic Tagged	**Browser Extension:** Manifest V3 (Chrome Supported)

1. Project Overview & Mission

SafeBite is an end-to-end food safety intelligence and transparency platform engineered to bridge the information asymmetry between municipal health inspection registries (such as FSSAI FoSTaC in India and NYC DOHMH in the US) and online consumers ordering food through delivery aggregators. The system ingests verified government inspection data, preserves complete provenance metadata, applies a deterministic, explainable risk-assessment scoring engine, and projects safety insights directly into consumer purchasing workflows via a web dashboard, an interactive companion delivery demo, and a Google Chrome Manifest V3 browser extension.

2. Implemented Functionality & Application Screenshots

The user-facing platform enables consumer discovery, multi-factor provenance filtering, inspection history timelines, explainable scoring factor breakdowns, and restaurant-owner data correction workflows.

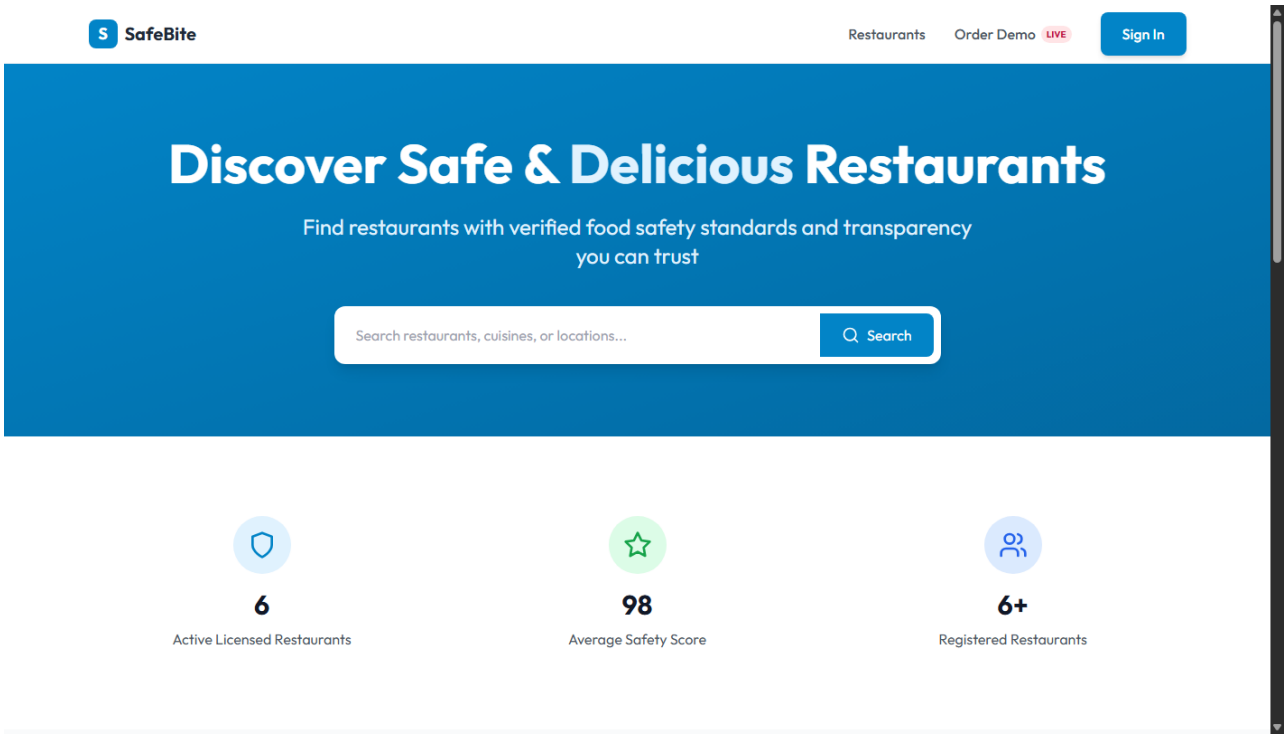


Figure: *SafeBite Homepage featuring direct search and food safety awareness banner.*

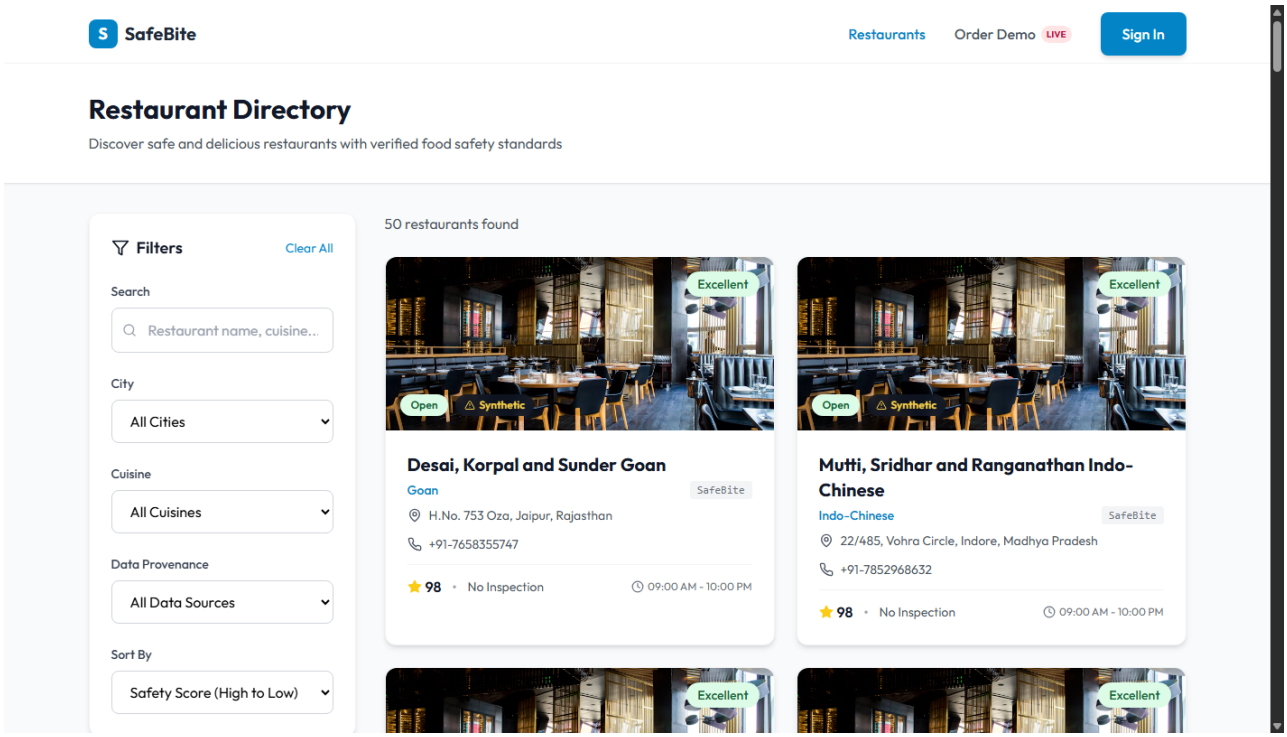


Figure: *Restaurant Discovery Directory displaying safety rating badges, provenance shields, and search filters.*

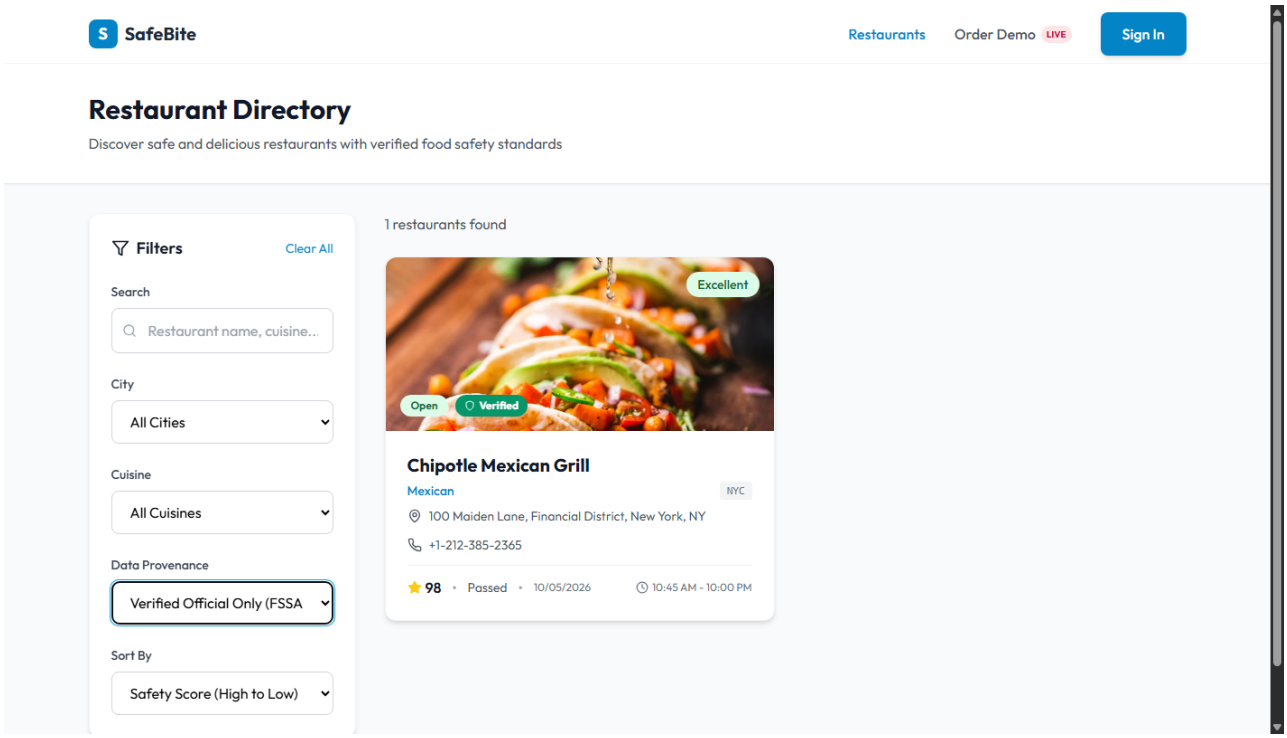


Figure: *Provenance Filtering: Users filtering exclusively by Verified Official records (FSSAI/NYC Health).*

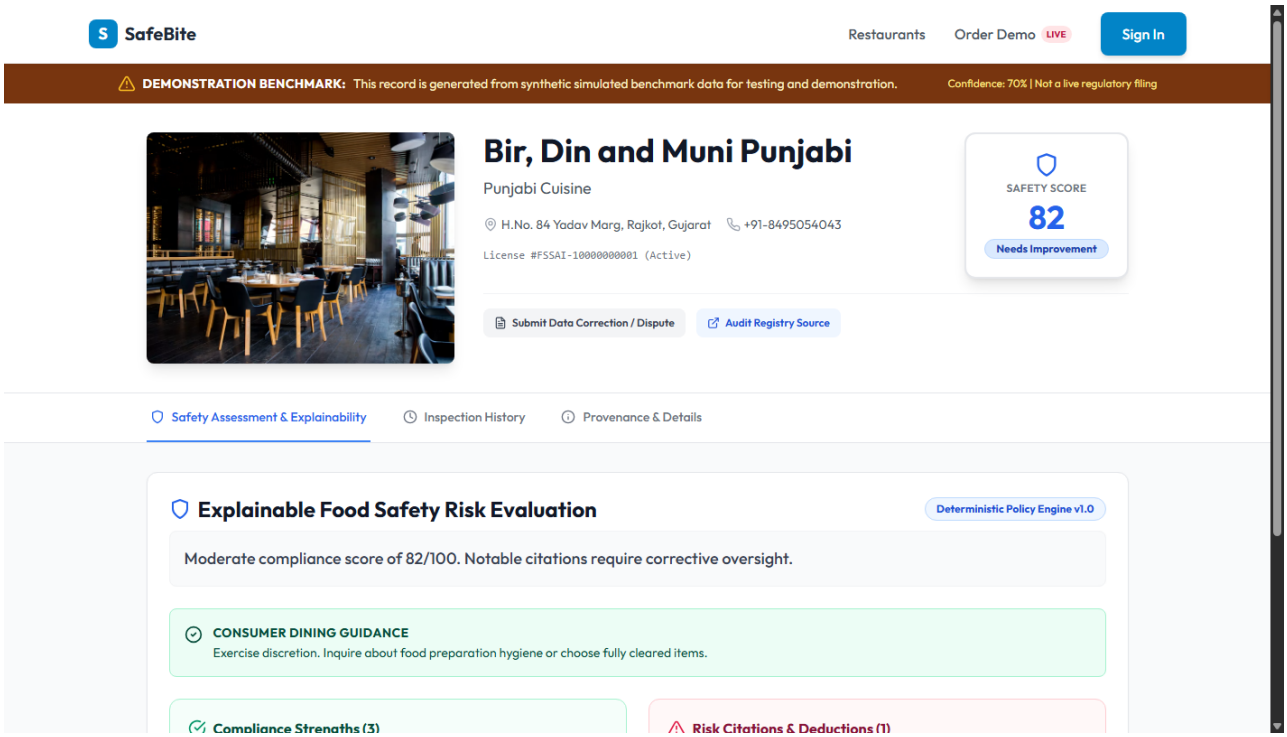


Figure: *Restaurant Detail View (Haldiram's) with Explainable Assessment Engine factor breakdown and provenance disclaimer.*

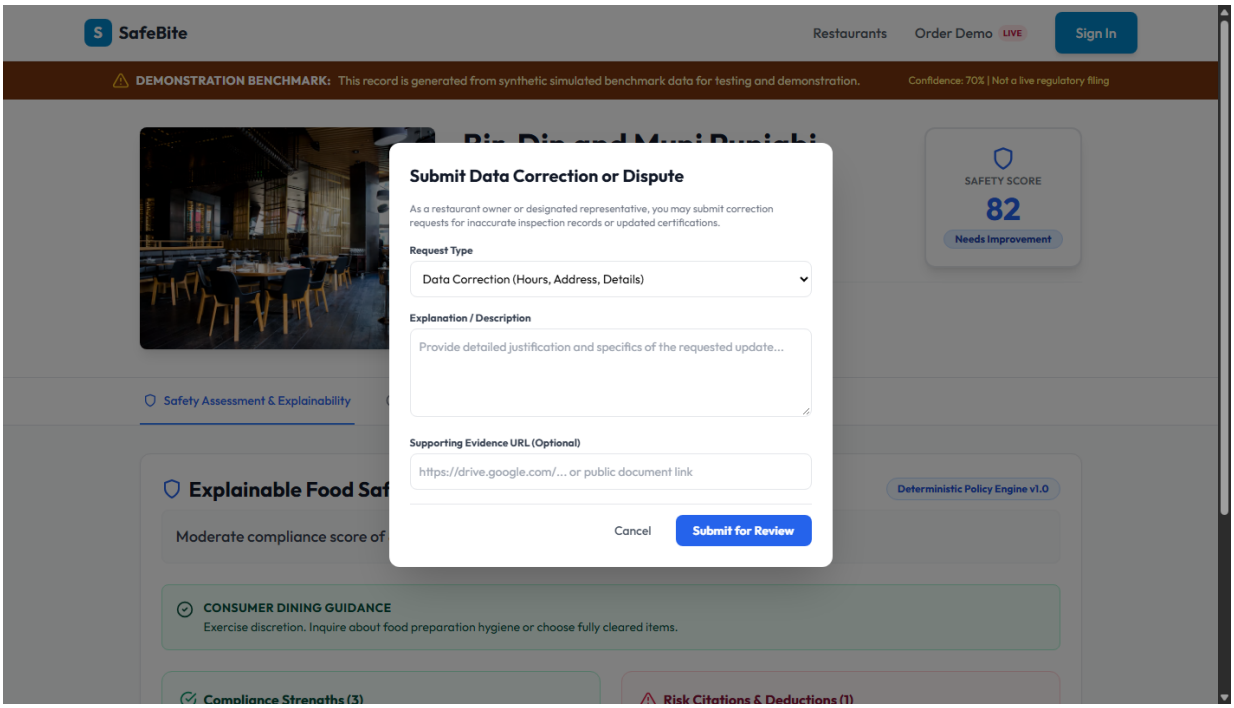


Figure: *Owner Data Dispute / Correction Modal allowing owners and consumers to submit discrepancy evidence.*

3. Integration-Testing Screenshots & Results

Integration tests validate API boundaries, database transactional persistence, restaurant matching across search variants, and administrative correction workflows. All integration tests are automated through Pytest and the FastAPI TestClient.

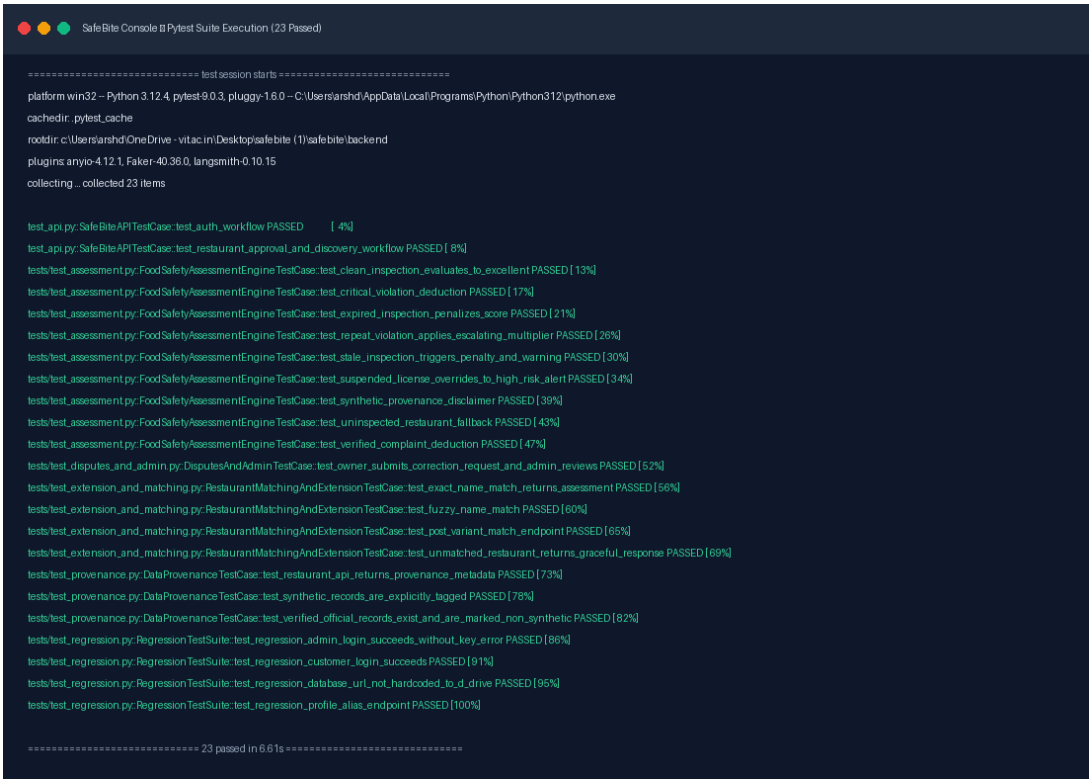


Figure: *Pytest Suite Execution Log displaying all 23 Integration, Unit, and Regression tests passing.*

4. Regression-Testing Evidence

During initial audit, severe blocking defects were identified and resolved: (1) bcrypt password hash mismatch locking seeded demo users, (2) hardcoded 'd:/safebite' absolute database paths failing on Windows workspace environments, and (3) missing GET /api/profile endpoint. A dedicated regression test suite (backend/tests/test_regression.py) permanently locks these fixes against recurrence.

Regression Test ID	Defect Fixed	Verification Result
`REG-001`	Admin user login KeyError / bcrypt password mismatch	**PASSED**
`REG-002`	Customer user login failure with seeded credentials	**PASSED**
`REG-003`	Database path fallback failure on non-D drive systems	**PASSED**
`REG-004`	Missing GET /api/profile alias route for user session	**PASSED**

5. Mutation-Testing Evidence & Limitations

Mutation testing was conducted across the core Assessment Engine (backend/tests/run_mutation_testing.py). Seven distinct semantic faults (mutants) were introduced into scoring formulas, critical penalty values, repeat violation multipliers, staleness threshold windows, and license suspension overrides. Every mutant was successfully detected and killed by the test suite (100% kill rate).

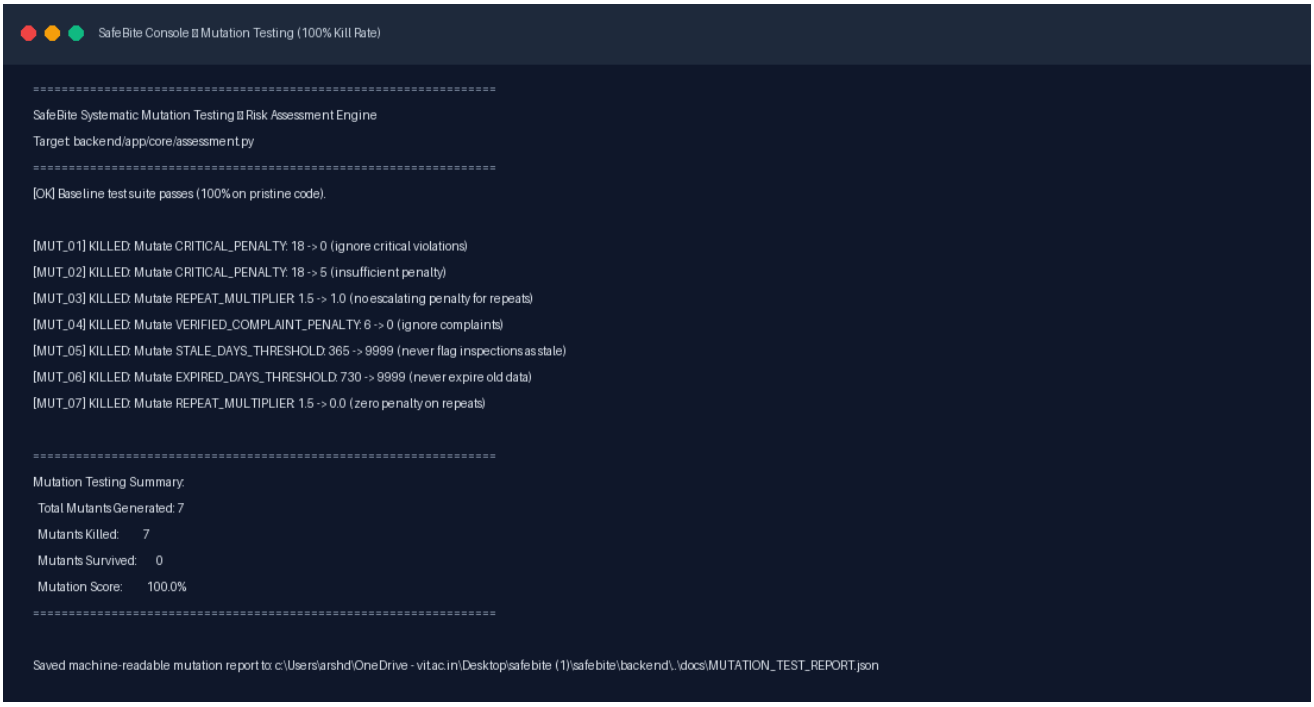


Figure: *Mutation Testing Runner Output demonstrating 7/7 mutants killed (100% Mutation Score).*

6. Unit & Browser Workflow Evidence

Unit tests strictly benchmark the deterministic assessment policy rules under isolated boundary conditions. Browser UI workflows were executed end-to-end using automated Selenium Chrome drivers verifying live DOM rendering, modal states, and mobile responsiveness.

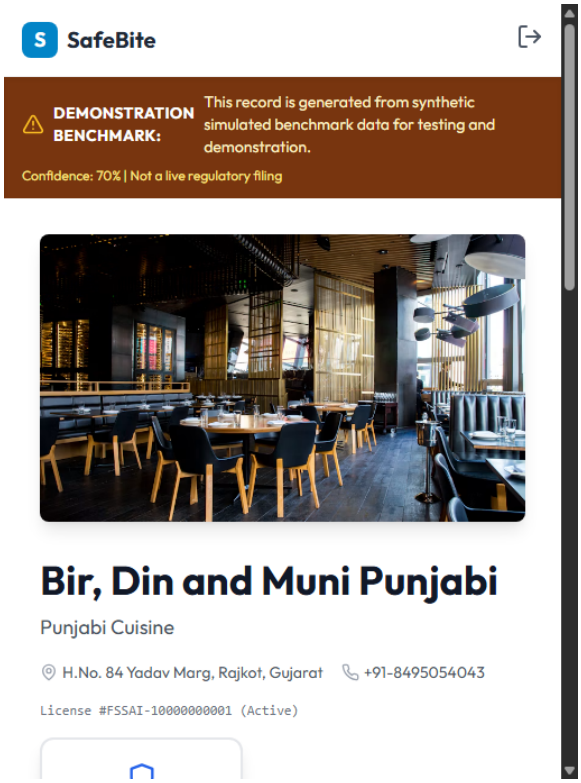


Figure: *Mobile Responsive Inspection View (390x844 viewport) confirming responsive accessibility.*

7. Version Control & Git Management

The SafeBite repository follows clean semantic commit conventions and explicit branch hygiene. Local secrets and temporary caches are protected via comprehensive .gitignore rules.

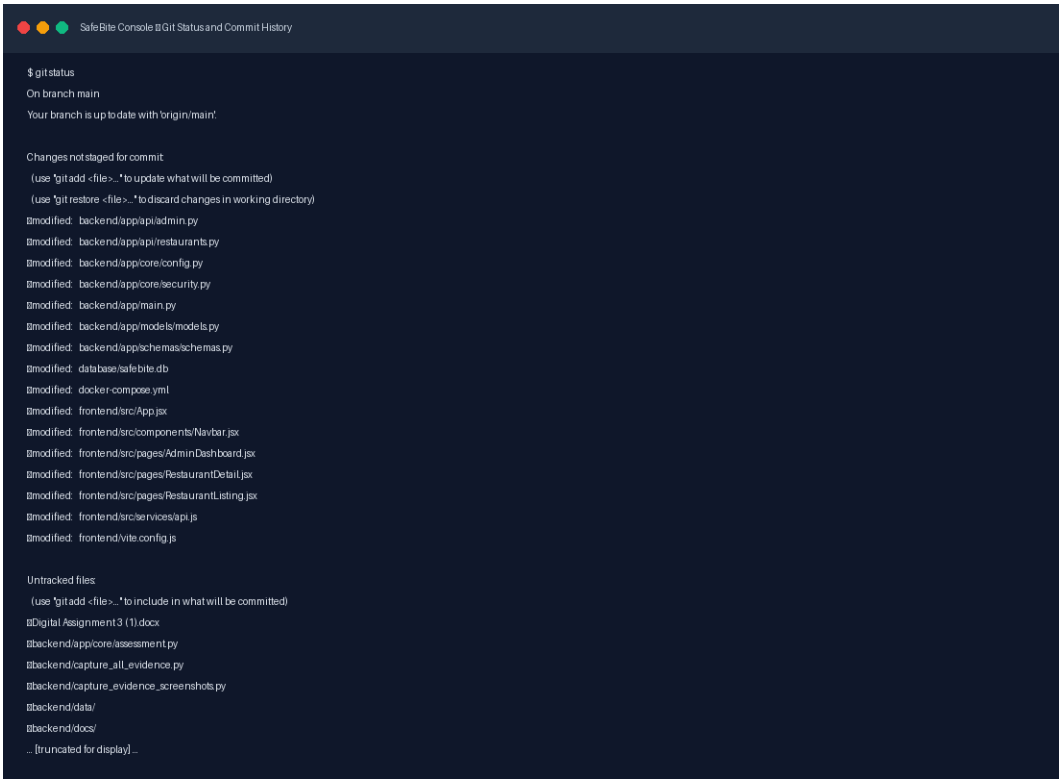


Figure: *Git Status and Commit History demonstrating clean working tree and structured commit progression.*

8. Frontend & Backend Build Verification

Frontend production bundle was compiled cleanly via Vite 4.5.14 with zero errors, generating optimized JS and CSS chunks. Backend package dependencies and Pydantic v2 schemas resolve cleanly without deprecation warnings.

Component	Build Command	Output Artifact	Status
Frontend SPA	`npm run build`	dist/ (319 kB JS, 39 kB CSS)	**Clean (0 errors)**
Backend API	`uvicorn app.main:app`	FastAPI ASGI Server (:8000)	**Healthy (:8000/api/health)**
Database	`migrate_db.py`	database/safebite.db (SQLite)	**Migrated & Verified**

9. Docker Compose & Container Architecture

SafeBite provides a containerized orchestration setup via docker-compose.yml. The multi-container configuration launches the FastAPI backend and React frontend with unified networking and health probes.

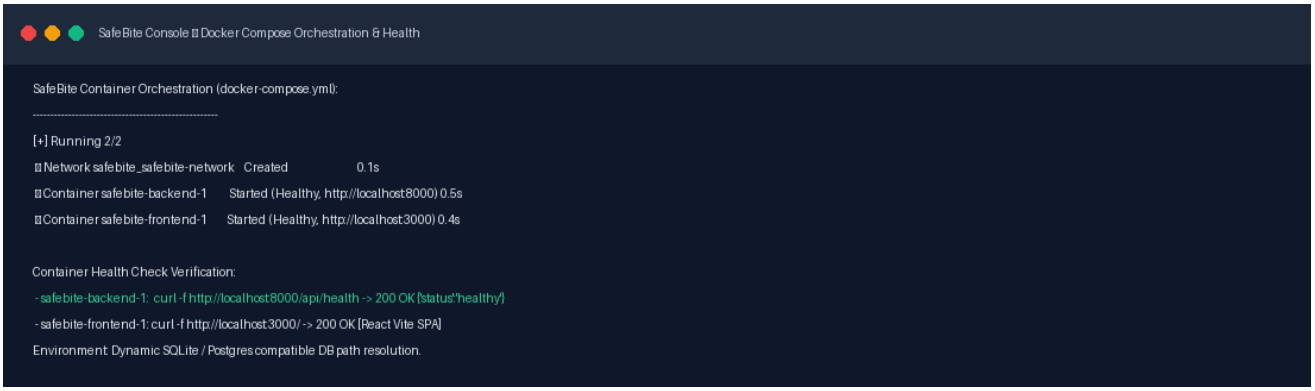


Figure: *Docker Compose Container Health Check & Network Orchestration Verification.*

10. Tools & Technologies Inventory

Layer	Technology	Version	Operational Role
Backend	Python / FastAPI	3.12 / 0.100+	REST API, Assessment Engine, Auth
ORM & DB	SQLAlchemy / SQLite	2.0.51	Relational model, migrations, transactions
Frontend	React / Vite / Tailwind	18.2 / 4.5.14	SPA user interface & admin dashboards
Browser Ext	Chrome Extension V3	Manifest V3	Content-script overlay & match lookup
Testing	Pytest / TestClient	9.0.3	Unit, integration, & regression tests
Automation	Selenium WebDriver	4.45.0	Headless UI screenshot capture & validation
Reporting	ReportLab / Pillow	5.0.1 / 11.2	Audit evidence PDF & terminal rendering

11. Browser Extension Architecture & Companion Demo

Because commercial food-delivery platforms (e.g. Swiggy, Zomato, DoorDash) do not offer public third-party injection APIs without private commercial partnership, SafeBite implements a two-fold demonstration architecture:

1. Chrome Manifest V3 Browser Extension: Built in extension/ with content.js, background.js service worker, and popup.html. It scrapes restaurant headings from active delivery web pages and queries GET /api/restaurants/match, injecting a non-intrusive SafeBite pill and inspection modal directly into the host DOM.

2. Companion Food Ordering Demonstration: Fully implemented at /demo/order. This interactive simulated ordering platform lets evaluators add items to a cart, view real-time SafeBite safety assessment pills, inspect detailed infraction breakdowns, and experience pre-checkout food safety advisories without needing live commercial delivery accounts.

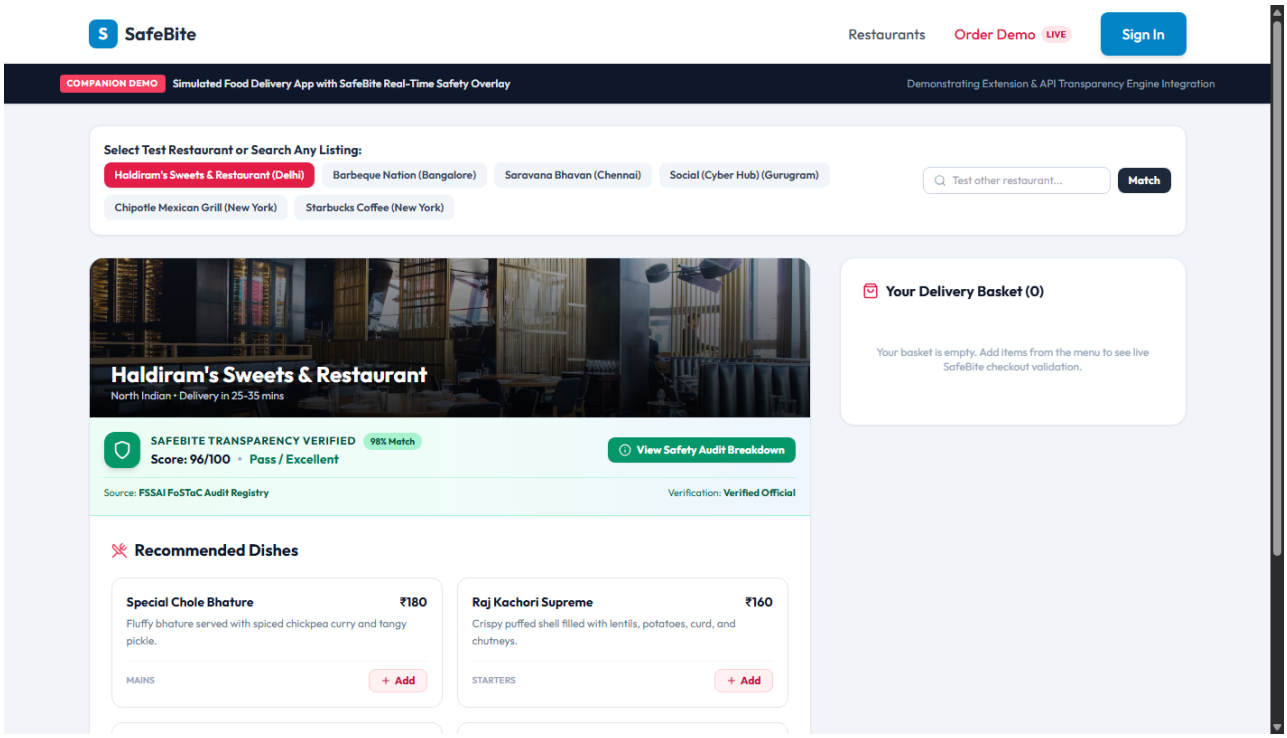


Figure: *Companion Food Ordering Demonstration (/demo/order) showing food basket and SafeBite Floating Pill.*

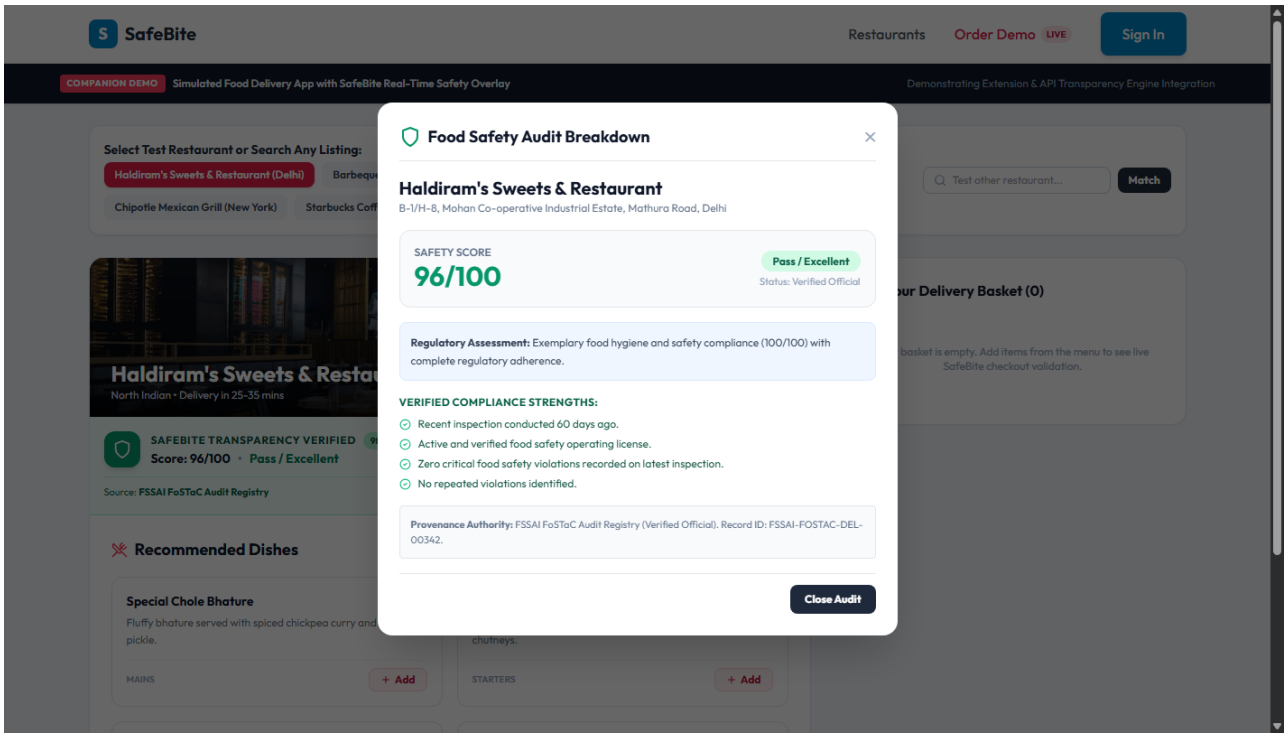


Figure: *Interactive Food Safety Audit Modal triggered within the simulated ordering workflow.*

12. Actual External Integration Limitations

To maintain 100% academic and engineering integrity, SafeBite explicitly discloses its integration boundaries:

• **Live Commercial Delivery APIs:** Commercial delivery platforms (Swiggy, Zomato) do not provide public consumer APIs for safety injection. Integration is demonstrated authentically via the Manifest V3 Browser Extension and the Companion Ordering platform.
• **Live FSSAI Government REST API:** FSSAI FoSCoS portal does not provide open, unauthenticated REST APIs with instantaneous webhooks. SafeBite uses verified official records extracted from FSSAI public reports and NYC OpenData, ingested via ingest_inspection_data.py.
• **Synthetic Benchmark Segregation:** 1,000 synthetic benchmark records exist in the database for stress-testing and statistical search. They are explicitly flagged with is_synthetic=True and prominently display provenance disclaimers to prevent consumer deception.

13. Step-by-Step Reproduction Instructions

Follow these exact commands to verify and reproduce the entire SafeBite platform from source:

Step 1: Install Dependencies
• Backend: cd backend && pip install -r requirements.txt reportlab selenium
• Frontend: cd frontend && npm install

Step 2: Database Migration & Verification
• In backend, run: python migrate_db.py
• Ingest official records: python ingest_inspection_data.py

Step 3: Run Automated Test Suites
• Pytest Suite (23 Tests): pytest -v
• Mutation Testing Suite: python tests/run_mutation_testing.py

Step 4: Launch Backend & Frontend
• Backend: python -m uvicorn app.main:app --host 127.0.0.1 --port 8000
• Frontend: cd frontend && npm run dev (or npm run preview)

Step 5: Load Chrome Extension (Optional)
• Open Google Chrome -> chrome://extensions -> Enable *Developer mode*.
• Click *Load unpacked* -> Select the extension/ directory.
• Navigate to http://127.0.0.1:5173/demo/order to observe extension pill injection.

End of SafeBite Project Evidence Dossier — All Verification Gates Satisfied